

Self-replicating seedbox servers using programmable money

EternalSeedBox

Matei Dogariu

Public thesis defense – 25 June 2026

Supervisor: Johan Pouwelse
Delft University of Technology – EEMCS



01

The question

Distribution without an operator

Centralized distribution is a single point of control

Mainstream content distribution concentrates operational control on one entity that owns both the hardware and the catalog.

- One operator decides what stays online; availability depends on its finances, its policies, and its legal exposure.
- Peer-to-peer protocols (BitTorrent, IPFS) distribute the *data* across many nodes — so takedown is hard — but the *hardware* is still owned and kept alive by humans.

The open gap

Can the **infrastructure layer itself** be operator-independent — leasing and paying for its own hardware, with no human running it after genesis?

Can a network of autonomous software agents, seeded by a single human-initiated genesis node and controlled only by inherited economic parameters, **survive** and **grow** without further human intervention?

Research question

Read as the hypothesis under test: two outcomes must *both* hold – **persistence** and **expansion** – with control reduced to inherited parameters and no human in the loop after genesis.

Falsifiable: the closed-system run is a prediction the data must meet.

Where this sits scientifically

Prior self-replicators ran on *simulated* resources:

- von Neumann's automata¹ and Ray's **Tierra**² — organisms competing for simulated CPU and RAM.
- DAOs and autonomic computing (MAPE-K) — decentralized control over virtual, quota-based resources.

Novelty: the first self-replicator on **real**, **market-bought infrastructure paid for with real money** — each child is a leased VPS.

 TU Delft

Three contributions:

- Engineering** — first self-replicating economic agent that buys real infrastructure on an open market.
- Method** — unmodified production code run against a replica Bitcoin network and a mock VPS marketplace.
- Empirical** — fleet dynamics and one heritable trait under selection: a prediction met, one disconfirmed.

25 June 2026 5

¹J. von Neumann (1966). *Theory of Self-Reproducing Automata*. Ed. by A. W. Burks. Urbana, IL: University of Illinois Press

²T. S. Ray (1991). "An approach to the synthesis of life". In: *Artificial Life II*. ed. by C. G. Langton, C. Taylor, J. D. Farmer, and S. Rasmussen. Redwood City, CA: Addison-Wesley, pp. 371–408

02

The apparatus

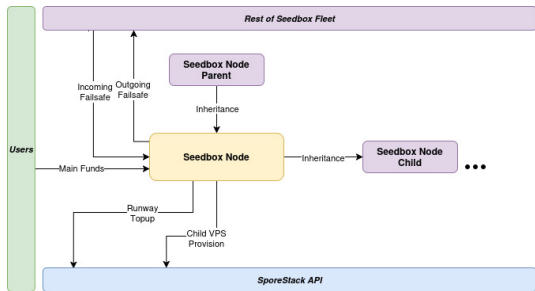
How a node lives, reproduces, and varies

System at a glance: one autonomous node

A node is one Linux process (the *Orchestrator*) bundling:

- VPS leased via **SporeStack** (a Bitcoin-native marketplace).
- **Bitcoin wallet** — pays its own bills.
- **libtorrent seedbox** — serves Creative Commons content.
- **IPv8 overlay** — gossips health telemetry to peers.
- **Decision logic** — the genome.

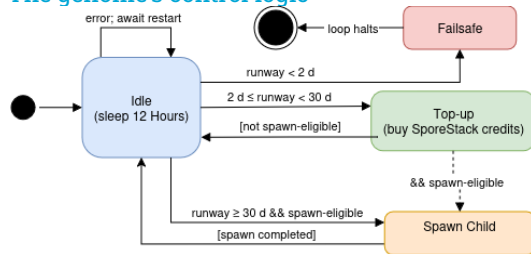
No central authority: genesis is the only human action; thereafter the fleet self-governs.



Arrows trace Bitcoin movements.

The decision loop

The genome's control logic



Every 12 h, the node takes the first action whose guard passes:

- Failsafe** — runway < 2 days → sweep funds to the healthiest peer.
- Top-up** — runway < 30 days → buy VPS credit (*doesn't return* — re-scans, can spawn the same tick).
- Spawn** — if eligible → provision + fund a child.
- Idle** — wait for the next tick.

Identical code runs in every node — no node is special.

Reproduction mechanics

The constraint against self-bankruptcy

Check runway after the spawn, not before

A node spawns only if its **post-spawn total runway** $\geq 60 \times (1 + \text{caution})$ days *and* its wallet can pay the child's VPS. Total runway = pre-paid VPS days + SporeStack credit + convertible Bitcoin.

- The child starts with a **30-day** pre-paid VPS and inherits **40%** of the Bitcoin remaining *after* the parent covers the child's invoice – a parent never commits to an inheritance it can't cover.
- Pipeline: mutate caution $\rightarrow$ provision VPS + fresh wallet $\rightarrow$ deploy code over SSH $\rightarrow$ transfer inheritance.

Caution — the single gene under selection.

One value $c \in [0.35, 0.9]$; the genesis node starts at $c = 0.5$. Everything else is inherited identically.

The heritable trait

It scales the spawn threshold
 $60 \times (1 + c)$:

caution c	threshold
0.35	81 days
0.5	90 days
0.9	114 days

Low caution = spawn sooner; high = retain more.

Mutation: $c_{\text{child}} = c_{\text{parent}} + \mathcal{N}(0, 0.05) + 0.2(0.5 - c_{\text{parent}})$,
clamped — Gaussian drift with mean reversion to 0.5.

How it actually evolves → Results

03

The method

What runs in the lab is the code that would run live

Faithful exercise of production code

Internal validity

Unchanged from live deployment

Entrypoint, orchestrator, Bitcoin wallet, and IPv8 community run **identically** to a real deployment – the only edits concern compressed-time intervals: 1 sim-day $\approx$ 22 s, a 90-sim-day runway $\approx$ 32 min.

Swapped – network layer only:

- Bitcoin $\rightarrow$ private **regtest** (1 s blocks), same Python library paths.
- SporeStack API $\rightarrow$ host-side **mock**, identical endpoints; invoices paid in real regtest BTC, mined locally.
- Each “server” = a real **Alpine LXC container** (own FS / PID tree / IP); parent deploys over the production SSH pipeline.
- Peer discovery via the same IPv8 tracker; a bootstrap container boots first, keeping the fleet off the public Internet.

What runs in the lab is the code that would run live.

Two experiments, one income knob

Closed system vs. open system

No-income run — closed system

- Genesis seeded with one **€10,000** lump sum; no exogenous income.
- Every node spends only its birth runway + inheritance.
- A **falsifiable closed-system prediction**: the fleet cannot sustain itself indefinitely.
- `faucet_enabled = false`

Income run — open system

- Host-side income pays the fleet; per-node income $\approx$ invariant (payout scales with node count).
- Uneven daily shares (Dirichlet draw).
- **Node cap 60**: income suspended at 60 nodes, resumed below 40 $\rightarrow$ alternating growth/drought.
- `faucet_enabled = true`

Honesty upfront

Under load the regtest network drops $\sim$ half of all broadcasts, so income is set at $\sim 10 \times$ **per-node rent** — well above any realistic seeding rate. **The income run tests fleet mechanics under favorable conditions, not economic viability.**

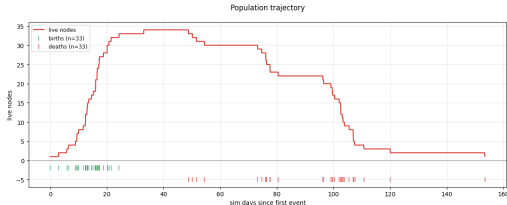
04

Results

One plot, one mechanism – a prediction met and one disconfirmed

No-income run: a fixed budget, predictably spent

Closed system

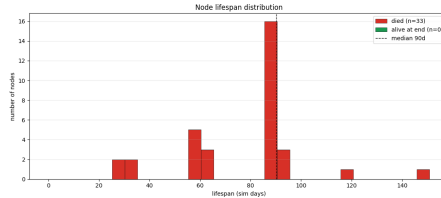


- With no income every €3.27 buys exactly one node-day, so the fleet's **total lifespan is fixed** at 3061 node-days (a mean near 93 days across 33 nodes).
- Births front-load while capital is concentrated; the fleet then only *loses* nodes.
- Long-lived survivors are **not luck** — the failsafe sweeps a dying node's balance to its richest peer.

The closed-system prediction holds: a once-funded fleet cannot self-sustain.

The clusters were predicted before the run

Lifespans, no income



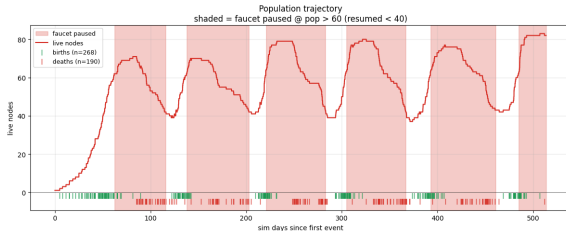
- Each cluster is a spawn constant fixed *a priori*: **90 d** is the post-spawn floor $60(1+c)$ at $c = 0.5$ – a band over 81–114 d as caution varies; **60 d** is one inheritance step above the starter; **30 d** is the bare starter runway.

Specification met

Theory fixes the cluster positions before any data exists; the histogram lands on them – the cleanest, most falsifiable result.

Income run: persistence, and an artifact

Open system

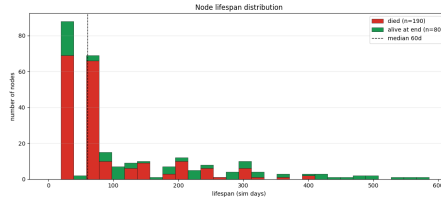


- The 40–80 oscillation is an **artifact of the node cap**: the host tops out near 80 containers, so income is cut at 60 nodes and restored below 40.
- Underneath hides a real self-limiting dynamic — each spawn spreads capital over more nodes, shrinking per-node surplus — but at these sizes the cap masks it (→ future work).

Income above per-node rent turns a 153-day die-off into persistence past 510 days.

Survival without longevity

Lifespans, with income



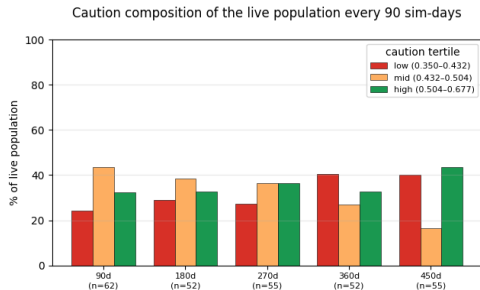
- Uneven early income (Dirichlet shares) splits the fleet: a node drawing above-median income early crosses the spawn threshold with a large surplus and keeps 60% of it, so **early wealth compounds** into a few long-lived lineages.

The turnover principle

Survival needs only *some* node holding a surplus, so a small well-funded minority replaces the short-lived majority – the fleet tolerates high turnover by design.

Caution under selection: the hypothesis

Trait distribution of live nodes, 90-day snapshots



Mutation is **mean-reverting toward 0.5**, yet the middle hollows while both extremes grow — polarization against a centering pull means **genuine selection pressure** is acting.

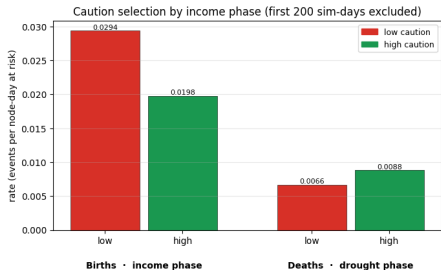
Hypothesis: the income on/off cycle rewards opposite strategies — low caution exploits growth (spawn aggressively), high caution survives drought (hoard runway), mid caution exploits neither.

Two falsifiable predictions:

- low caution reproduces faster during income;
- high caution dies less during drought.

Caution under selection: the result

Birth and death rates by caution group and phase



Prediction 1 – confirmed

Low caution reproduces ~50% faster while income flows (0.0294 vs. 0.0198 births/day).

Prediction 2 – disconfirmed

No survival edge – high caution even dies slightly *more* in drought (0.0088 vs. 0.0066 per day).
Selection favored low caution: the reproduction gap ran in every phase.

The environment was **income-dominated** – pauses too short for caution to pay off. The mechanism works; testing the survival half awaits sustained scarcity (→ future work).

05

Viability & validity

Economics, and the limits of the evidence

Not cost-competitive – and that is the point

Storage economics

Cost per TB-month

EternalSeedBox	€241
Commercial object storage	€20–25
Premium	≈ 10×

Commercial = object-storage list price.³

Break-even **€3.27 / node / day** (€98 per 30 d, 400 GB served).

A deliberate niche, not a general store

Each node pays **retail VPS** – no economies of scale, so the rate cannot match commercial storage. The premium buys availability that **survives takedown** and open-source accountability. The market is users whose content no one else will host, funding the infrastructure directly.

Competitive only where availability itself is the product.

³Amazon Web Services (2026). *Amazon S3 Pricing*. Amazon Web Services, Inc. URL: <https://aws.amazon.com/s3/pricing/>

What the runs establish – and what they cannot

Threats to validity

Internal validity – strong

- Unmodified production code against faithful **Bitcoin** and **VPS-marketplace** replicas.
- Replication verified under **both** the closed and income conditions.
- The closed-system prediction was met.

External validity – open

- Node cap 60 – host ceiling ($\sim 60-80$ containers), not economics.
- Regtest drops $\sim 50\%$ of tx $\rightarrow$ income set $10\times$ rent $\rightarrow$ economy inflated.
- Single VPS provider (SporeStack) – no failover.
- Revenue mechanism not yet designed.
- Fixed genesis catalog.

The mechanism is validated; viability is not – by construction.

06

Conclusion

The answer, and what comes next

Yes — conditional.

A fleet sustains and grows itself after genesis **when its income covers the per-node rent.**

The answer

Three contributions:

- a. **Engineering** — first self-replicating economic agent that buys real infrastructure on an open market.
- b. **Method** — unmodified production code against faithful Bitcoin + VPS-marketplace replicas.
- c. **Empirical** — fleet dynamics and one heritable trait under selection: a prediction met, one disconfirmed.

Thank you

Questions welcome

Future work

- A **payment incentive** — make consumers fund seeding directly (the missing revenue layer the rest of the system already supports).
- **Decentralized moderation** — resolve removal requests across a fleet with no central authority (prerequisite for user-contributed content).
- **Long drought, no node cap** — to test the unconfirmed survival half of the caution hypothesis.

Bibliography I

 Amazon Web Services (2026). *Amazon S3 Pricing*. Amazon Web Services, Inc. URL: <https://aws.amazon.com/s3/pricing/>.

 Neumann, J. von (1966). *Theory of Self-Reproducing Automata*. Ed. by A. W. Burks. Urbana, IL: University of Illinois Press.

 Ray, T. S. (1991). "An approach to the synthesis of life". In: *Artificial Life II*. Ed. by C. G. Langton, C. Taylor, J. D. Farmer, and S. Rasmussen. Redwood City, CA: Addison-Wesley, pp. 371–408.